TMS — Campus Ticket & Complaint Management System

Complete Viva Voice & Presentation Script (Simple Spoken English + Technical Q&A)

MERN Stack (MongoDB, Express, React, Node.js) | React Native Mobile (Expo SDK 58) | JWT + RBAC Authentication | Cloud MongoDB Atlas

1. The 60-Second Presentation Pitch (How to Start)

Memorize or speak these exact lines when the examiner asks: *"Introduce yourself and explain your project."*

🗣️ **SPEAK THIS IN VIVA:**

"Good morning respected examiners. My project is **TMS — Ticket Management System**. It is a full-stack, cross-platform campus facility maintenance system built using the **MERN stack** and **React Native Expo**.

In colleges and workplaces, reporting infrastructure issues like damaged computers, Wi-Fi connectivity problems, electrical faults, or plumbing issues is traditionally done through paper registers or manual phone calls. This results in lost complaints, long delays, and no accountability.

To solve this, our project delivers an end-to-end digital platform where students and staff can raise a ticket in under 30 seconds with photos, administrators can assign staff with one click, and technicians can resolve and close tickets with real-time status tracking."

💡 **Tamil Tip:** ஆரம்பிக்கும் போது பதட்டப்படாமல் இந்த 3 விஷயங்களை சொல்லுங்கள்: 1. Problem (Manual registers delay complaints), 2. Solution (Digital ticketing web + mobile app), 3. Tech Stack (MERN + React Native).

2. Problem Statement vs. Our Solution

Traditional / Existing System	Our Proposed TMS System
Manual register books or verbal complaints.	**Digital 24/7 Portal** accessible via Web and Android/iOS Mobile.
Complaints get lost or ignored without proof.	**Evidence Upload** (Photos/Documents attached to tickets).
No transparency on who is fixing the issue.	**Role-Based Assignment** (Assign to Electrician, Plumber, Network staff).
No tracking of resolution time or user feedback.	**Live Status Lifecycle** (Pending → Assigned → In-Progress → Resolved).

3. Technology Stack & Why We Chose It

Layer	Technology	Why We Used It (Examiner Question)
Web Frontend	React 18, React Router v6, Axios, Vanilla CSS	Fast single-page application (SPA), reusable components, dynamic dashboards.
Mobile App	React Native, Expo SDK 58	Single codebase for Android and iOS, hardware photo attachments, cross-platform storage.
Backend API	Node.js & Express.js	Non-blocking asynchronous REST API, high concurrency, seamless JSON handling.
Database	MongoDB Atlas (Cloud NoSQL)	Flexible schema, JSON document storage, high scalability, 24/7 cloud availability.
Security	JWT (JSON Web Token) & bcryptjs	Stateless authentication with 7-day token expiry, salted password encryption.

4. Role-Based Access Control (RBAC)

Our system strictly manages permissions across 3 tiers of users:

1

SuperAdmin

Full campus authority. Manages departments, blocks, rooms, users, assigns staff, and exports reports.

2

Field Technicians

Electrician, Plumber, Network, Hardware. Views assigned tickets and updates status to In-Progress / Resolved.

3

End Users

Students, faculty, staff. Raises tickets with photo evidence, tracks status, and submits 1-5 star ratings.

5. Ticket Lifecycle (How It Works)

1

Pending

User creates ticket with Block, Room & Category.

2

Assigned

Admin reviews and assigns ticket to specialized staff.

3

In-Progress

Staff inspects location and begins repair work.

4

Closed / Resolved

Work completed. User submits rating and feedback.

6. Most Frequently Asked Viva Questions & Answers

Q1: What is the main purpose of this project?

Answer: The main purpose is to automate and streamline campus facility maintenance by replacing paper-based complaints with an online ticketing platform supporting real-time tracking, photo attachments, and staff assignments across web and mobile.

Q2: Why did you choose MERN stack instead of traditional PHP/MySQL or Java?

Answer: MERN uses JavaScript end-to-end across frontend, backend, and database queries. It provides high performance with Node's non-blocking I/O, rapid UI updates with React's Virtual DOM, and schema flexibility with MongoDB.

Q3: How is authentication handled in your project?

Answer: We use **JWT (JSON Web Tokens)**. When a user logs in with email and password, the server validates password using `bcrypt.compare()`. If valid, the server signs a JWT containing the user ID, email, and role. The client stores this token in localStorage/SecureStore and sends it in the HTTP `Authorization: Bearer <token>` header for subsequent API requests.

Q4: How do you handle password security?

Answer: Passwords are never stored in plain text. We hash them using `bcryptjs` with 10 salt rounds before saving to MongoDB. Even database administrators cannot see the raw passwords.

Q5: What database collections (models) do you have?

Answer: We have 6 main collections in MongoDB: **User** (credentials, role, department), **Complaint** (ticket title, category, block, room, status, priority, attachment), **Department**, **Programme**, **Block**, and **Room**.

Q6: How do file uploads work for ticket attachments?

Answer: We use **Multer** middleware on Express. Multer intercepts multipart/form-data requests, renames the uploaded image with a unique timestamp, and stores it in the server's `/uploads` directory while saving the relative file path in the MongoDB document.

Q7: How does the Mobile App communicate with the Backend?

Answer: The React Native mobile app communicates with the Node.js backend using **Axios** HTTP requests pointing to our REST API endpoint (e.g. `https://tms-1-ful8.onrender.com/api`). It stores the session token using **Expo SecureStore** on mobile and **localStorage** on web.

Q8: What is Expo SDK 58 and why did you use it?

Answer: Expo is a modern React Native framework that provides native modules for camera, file system, document picker, and secure storage without needing complex native Xcode or Android Studio configurations. SDK 58 uses the latest React Native 0.88 with improved performance.

Q9: What challenges did you face during development?

Answer: Three main challenges: 1. Configuring cross-platform storage so that token storage works on both mobile native hardware and web browsers. 2. Managing cascading dropdowns (selecting a department filters the blocks, which then filters rooms). 3. Protecting routes using middleware so unauthorized users cannot access SuperAdmin management panels.

Q10: What are the future enhancements for this project?

Answer: Future enhancements include push notifications via Firebase Cloud Messaging, AI-based automatic complaint classification, SLA (Service Level Agreement) breach countdown timers, and QR code stickers on classroom doors for instant room ticket scanning.

7. Live Demo Steps During Viva (Follow in Order)

1. **Step 1:** Open Web Dashboard at `http://localhost:3000` or Mobile App.
2. **Step 2:** Login as **SuperAdmin** (`admin@example.com` / `Passw0rd!`).
3. **Step 3:** Show the metrics overview (Total complaints, Pending, Assigned, Resolved).
4. **Step 4:** Show Master Data (Departments, Blocks, Rooms).
5. **Step 5:** Login as a Student / User (`user@example.com`).
6. **Step 6:** Raise a new complaint (Select CSE → IT Block → LAB-101 → Hardware Issue).
7. **Step 7:** Switch back to SuperAdmin → Assign complaint to Technician.
8. **Step 8:** Login as Technician (`Electrician@example.com`) → Change status to **Closed**.